

Dynamic UAV Routing: A Controlled Comparison of Naive Dijkstra Replanning and Reinforcement Learning

*Simra Imran

School of Electrical Engineering and Computer Science (SEECs)
National University of Sciences and Technology (NUST)
Email: simraimran158@gmail.com

*Muzzammil Sajid

Faculty of Mechanical Engineering (FME)
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology (GIKI)
Email: muzzammilsajid1@gmail.com

Abstract—Unmanned aerial vehicles (UAVs) operating in dynamic environments must route around obstacles that can appear, disappear, or move during flight. Classical graph-search methods such as Dijkstra’s algorithm provide optimal paths on a known static graph, but must be recomputed when the graph changes; reinforcement learning (RL) offers an alternative policy-based approach that reacts through learned action selection rather than explicit search. This paper presents a controlled, paired comparison of a naive Dijkstra replanning baseline and a DQN policy trained through curriculum learning, using a shared 15×15 grid-world UAV routing environment with identical movement and obstacle rules for both methods. We first validate the DQN policy on a static 40-scenario evaluation (obstacle density 0.2), where Dijkstra succeeds on 100% of scenarios against 97.5% for DQN, with a mean path-cost penalty of 0.92 units above optimal. We then evaluate both methods on a paired 50-scenario dynamic evaluation with three shared toggling obstacles, where Dijkstra succeeds on all 50 scenarios against 49 for RL, and produces significantly shorter or equal-cost paths on every paired comparison (Wilcoxon signed-rank $W = 0$, $p = 5.64 \times 10^{-7}$). The anticipated compute-time advantage of RL did not reach significance in the dynamic case ($p = 0.063$), though the static evaluation shows a clear asymmetry: Dijkstra pays one graph search per episode while DQN pays a forward-pass cost at every step, making DQN roughly four times slower in the static setting, a gap that narrows once Dijkstra must itself replan repeatedly under dynamic conditions. These results support an honest trade-off framing: RL demonstrates substantial adaptability, but naive Dijkstra replanning remains stronger in path optimality and reliability at the tested scale.

Index Terms—UAV routing, Dijkstra algorithm, reinforcement learning, dynamic obstacles, path planning, grid world

I. INTRODUCTION

Unmanned aerial vehicles (UAVs) increasingly operate in environments where routing conditions can change during flight. Obstacles may appear or disappear, restricted regions may change, and the route selected at launch may no longer be

valid later in the mission. This makes dynamic path planning a central problem for autonomous UAV navigation [1], [2].

Classical graph-search algorithms remain a strong baseline for this problem because they are deterministic, interpretable, and optimal when the graph and edge costs are known. Dijkstra’s algorithm, in particular, returns a shortest path when all edge weights are non-negative. However, in a dynamic environment, each obstacle change can invalidate the previous path and require replanning from the UAV’s current position. This repeated recomputation is correct but may become expensive as environment size and change frequency increase.

Reinforcement learning (RL) provides a different approach. Rather than recomputing a path through explicit graph search, an RL policy maps observations to movement actions after training. This can potentially reduce deployment-time computation and allow reactive behavior under changing conditions. However, learned policies may be suboptimal, may fail in some scenarios, and do not provide the same optimality guarantees as Dijkstra [3]–[5].

Before testing RL’s adaptability under dynamic conditions, it is necessary to establish that the trained policy performs reasonably well when the environment does not change at all; a policy that cannot approach optimal performance on a static grid is not a meaningful comparison point once obstacles start moving. We therefore evaluate both methods twice: first on a static 40-scenario benchmark that establishes a baseline for path-cost and compute-time behavior, and then on the paired 50-scenario dynamic benchmark that is this paper’s primary contribution.

This paper asks whether a dynamically trained RL policy can match or outperform naive Dijkstra replanning in a controlled UAV grid-routing environment. We compare the two methods on success rate, path cost, and compute time, both when the environment is static and when it changes during flight.

The main contributions are:

*These authors contributed equally to this work.

- a shared 15×15 UAV grid environment with identical movement and dynamic-obstacle rules for both methods;
- a static baseline evaluation validating the trained RL policy’s path quality and compute-time behavior before dynamic obstacles are introduced;
- a naive Dijkstra replanning baseline that recomputes after each dynamic obstacle change;
- a dynamically trained RL evaluation on the same 50 scenario IDs and start-goal pairs;
- a paired statistical comparison using Wilcoxon signed-rank testing.

II. RELATED WORK

Classical UAV path planning methods include graph-search algorithms, sampling-based planners, and potential-field approaches. Graph-search methods such as Dijkstra and A* are widely used because they are interpretable and provide strong guarantees when the environment is represented as a known graph with non-negative edge weights. Sampling-based planners such as RRT and PRM can explore larger spaces, but their randomized nature means they do not necessarily return optimal paths and can struggle in cluttered environments [1], [6]. This project uses Dijkstra rather than a sampling-based method as the classical baseline because it is deterministic, optimal on a static graph, and has well-characterized computational complexity, properties that make the trade-off being measured easier to state precisely.

Recent surveys of UAV path planning emphasize that classical planners remain important reference points because of their completeness, optimality, and predictable computational behavior [1]–[3]. Their main limitation in dynamic environments is that they generally assume a known graph. When obstacles or edge costs change, the planner must update or recompute a route.

Reinforcement learning has been explored as an alternative for UAV routing and obstacle avoidance. RL methods learn policies through interaction rather than explicit shortest-path search. Deep RL methods such as DQN are particularly relevant when the state space becomes too large for tabular Q-learning [4], [5], [7]. An improved Q-learning variant with revised reward and Q-value update rules has been shown to reduce computation time by 50% and shorten path length by 30% compared to a standard SARSA baseline on a UAV path-planning task [4]. Prior work reports that RL can improve autonomy and adaptability in complex settings, but it often trades away the guarantees of classical methods.

Dynamic and online UAV routing is especially relevant to this project. Rocha et al. propose dynamic Q-planning for online UAV path planning in unknown and complex environments, comparing reinforcement-learning-based planning with classical alternatives [8]. This supports the motivation for testing learning-based adaptation against a deterministic replanning baseline.

Much of the RL-versus-classical UAV literature compares methods across different metrics, environments, and hardware, making head-to-head comparison across papers difficult; few

evaluate both approaches on the exact same graph, obstacle layout, and start-goal pairs with a paired statistical test on the difference. This project fits into the literature by closing that specific gap: a controlled same-scenario comparison rather than a broad benchmark. Both methods are evaluated on the same grid, same dynamic cells, and same 50 start-goal pairs. This paired design allows statistical testing of path-cost and compute-time differences instead of relying on anecdotal comparisons.

III. METHODOLOGY

A. Environment Model

The UAV operating area is represented as a 15×15 grid. Each traversable cell is a node, and the UAV can move in eight directions: north, south, east, west, and the four diagonals. Orthogonal moves have cost 1.0, while diagonal moves have cost $\sqrt{2}$. All edge costs are non-negative, making Dijkstra’s algorithm valid for shortest-path computation.

Coordinates are represented as (row, col) , with $(0, 0)$ at the top-left corner. A movement is valid only if the destination cell is inside the grid and not blocked. Both Dijkstra and RL use the same movement rules to prevent inconsistencies between the two implementations.

B. Dynamic Obstacles

The dynamic environment uses fixed-position obstacles that toggle between blocked and passable states. For the paired dynamic evaluation, static obstacle density is set to 0.0 on both sides. This removes seeded static-layout mismatch between the Dijkstra and RL environments. The only changing cells are the three shared dynamic cells $(4, 4)$, $(8, 8)$, and $(12, 11)$.

Neither method receives advance visibility into future toggles. Both methods observe only the current grid state after a change occurs. This keeps the Dijkstra baseline genuinely naive and ensures that the RL policy is not given predictive information unavailable to the classical planner.

C. Dijkstra Replanning Baseline

The classical baseline applies Dijkstra’s algorithm to the current grid graph. In the dynamic case, whenever the environment changes, the planner recomputes a full shortest path from the UAV’s current position to the goal. This naive replanning strategy is intentionally simple: it is correct and interpretable, but can pay the full graph-search cost multiple times during a route.

With a binary heap priority queue, Dijkstra’s algorithm has time complexity $O((V + E) \log V)$, where V is the number of free cells and E is the number of valid movement edges. In this grid, each cell has at most eight neighbors, so E grows approximately linearly with V .

D. Reinforcement Learning Policy

The RL method uses the same eight movement actions as the Dijkstra environment. The dynamic RL model is trained through curriculum phases, beginning with static routing and moving toward the full three-obstacle dynamic configuration.

The policy uses local grid observations and positional information to select movement actions. Unlike Dijkstra, the RL policy does not explicitly search the graph at evaluation time.

E. Compute Time Measurement

Both methods report compute time as the total decision-making time required to traverse a full route, not per-decision latency. For Dijkstra, this is the cumulative wall-clock time across all `dijkstra()` calls made during a run: one initial call plus one call per dynamic change event. For the RL policy, this is the sum of wall-clock time for every `.predict()` call made during an episode; environment stepping, reward computation, and observation construction are excluded from the timer.

Because Dijkstra resolves a route in one or a small number of full graph searches while the RL policy makes one lightweight forward pass per step, the two numbers are not directly comparable as per-decision latencies. The reported totals are comparable only as route-level decision cost: how much compute each method consumed to get from start to goal in that scenario. A single warm-up inference call is discarded before timing begins to exclude first-call framework initialization overhead from the RL measurements.

IV. EXPERIMENTAL SETUP

A. Static Evaluation

The static evaluation uses 40 paired start-goal scenarios on the same 15×15 grid, with obstacle density 0.2 and seed 42. Start-goal pairs are drawn from the free cells of a single `GridEnvironment` instance and validated against Dijkstra on that same grid; the DQN evaluation environment receives the identical obstacle layout via direct injection rather than independent seeding. No dynamic obstacles are present in this evaluation.

B. Dynamic Evaluation

The paired dynamic evaluation uses 50 matched scenarios. Each scenario has a fixed scenario ID, start cell, and goal cell. The same scenario list is used for both Dijkstra and RL evaluation. Pairing is important because comparing unrelated random episodes would confound method performance with scenario difficulty.

The evaluation configuration is:

- grid size: 15×15 ;
- movement: 8-connected;
- orthogonal movement cost: 1.0;
- diagonal movement cost: $\sqrt{2}$;
- static obstacle density: 0.0;
- dynamic cells: (4, 4), (8, 8), and (12, 11);
- number of scenarios: 50.

The measured metrics are success rate, realized dynamic path cost, compute time, and paired path-cost difference. Statistical significance is evaluated using the Wilcoxon signed-rank test on paired successful scenarios.

V. RESULTS

A. Week 2: Static Environment

The static evaluation used 40 paired start-goal scenarios on a 15×15 grid with obstacle density 0.20 and seed 42. Start-goal pairs were drawn from the free cells of a single `GridEnvironment` instance and validated against Dijkstra on that same grid. The DQN evaluation environment had the same 15×15 obstacle layout as the Dijkstra environment, achieved by directly copying `simra_env.blocked` into the DQN grid after construction; this is a stronger guarantee than matching seeds, as it carries no dependency on RNG implementation details agreeing between the two environment classes. Dijkstra computed a single shortest-path query per scenario; the DQN policy ran a full episode from each start to goal.

TABLE I
WEEK 2 STATIC EVALUATION SUMMARY (40 SCENARIOS).

Metric	Dijkstra	DQN
Success rate	40/40 (100%)	39/40 (97.5%)
Mean path cost	—	+0.9248 above Dijkstra
Max suboptimality gap	—	+3.6569
Mean route-level compute time	0.78 ms	3.20 ms (successes only)

DQN matched Dijkstra exactly (gap = 0) on 14 of 39 successful scenarios and produced a higher path cost on the remaining 25. The single failure (start (12, 0), goal (13, 10)) was a step-limit timeout at 225 steps with no collision, indicating the policy entered a loop rather than crashing. The compute-time figures are route-level totals as defined in Section III-E: one Dijkstra call per scenario versus accumulated `.predict()` calls per DQN episode.

B. Week 3: Dynamic Environment

The dynamic evaluation used 50 paired start-goal scenarios sharing the same scenario IDs and start-goal pairs across both methods. Static obstacle density was set to 0.0 on both sides; the only changing cells were the three shared dynamic obstacles at (4, 4), (8, 8), and (12, 11), each toggling on a five-step period. Dijkstra succeeded in all 50 scenarios; the RL policy succeeded in 49 of 50. The failed RL scenario is included in success-rate reporting but excluded from paired path-cost statistics because no valid RL path cost exists.

1) *Success Rate*: Figure 1 shows the success rate for both methods. Dijkstra achieved 100% success, while RL achieved 98% success.

2) *Path Cost*: Path-cost statistics are computed on the 49 scenarios where both methods succeeded. Dijkstra was never more expensive than RL: RL matched Dijkstra in 16 scenarios and produced a higher path cost in 33 scenarios. The mean path-cost difference, measured as RL minus Dijkstra, was +1.369441, and the median difference was +0.828427.

A Wilcoxon signed-rank test on paired path-cost differences gave $W = 0$ and $p = 5.64 \times 10^{-7}$. This result is statistically significant, indicating that Dijkstra produced

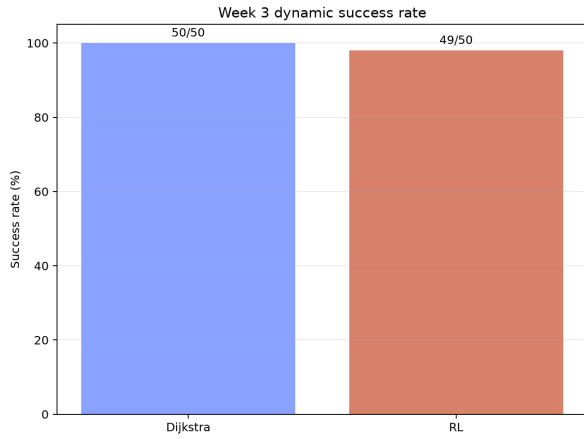


Fig. 1. Dynamic-routing success rate over 50 matched scenarios.

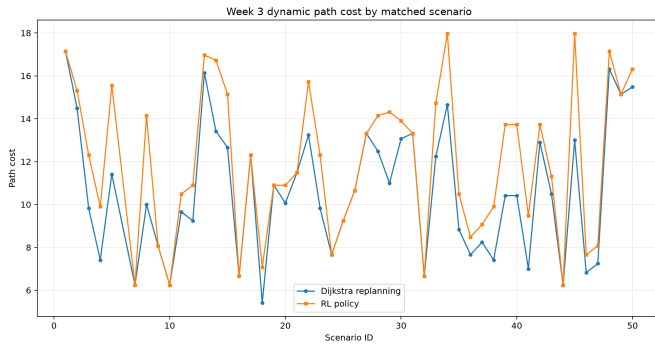


Fig. 2. Dynamic path cost by matched scenario for Dijkstra replanning and RL.

significantly shorter or equal paths than RL in the tested dynamic environment.

3) *Compute Time*: Mean cumulative Dijkstra planning time was 4.627753 ms, while mean RL evaluation time was 5.334265 ms on the paired successful scenarios. The Wilcoxon p-value for compute-time difference was 0.062865, which is not statistically significant at $p < 0.05$.

VI. DISCUSSION

The results show that naive Dijkstra replanning remains the stronger method on the current controlled benchmark. Dijkstra succeeded on every scenario and produced shorter or equal paths than RL on all paired successful scenarios. The RL policy still demonstrates substantial adaptability, completing 49 out of 50 dynamic scenarios under the same obstacle schedule and scenario pairing. However, these results do not support a claim that RL outperforms Dijkstra in this setup.

The compute-time result is also important. A common motivation for RL is that a trained policy may avoid repeated graph search during deployment. In this experiment, however, the measured compute-time difference was not statistically significant. This likely reflects the small grid size: a 15×15 graph is small enough that repeated Dijkstra replanning remains cheap.

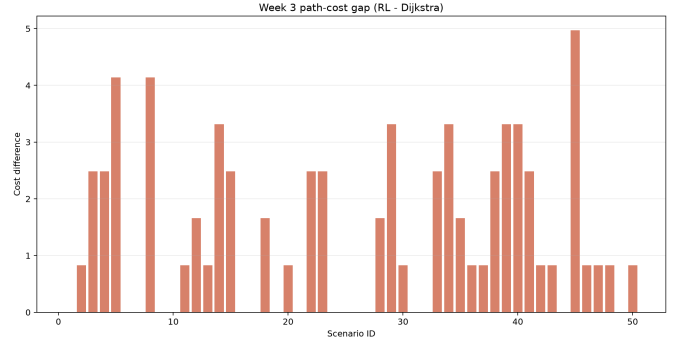


Fig. 3. Path-cost gap measured as RL minus Dijkstra. Positive values indicate higher RL path cost.

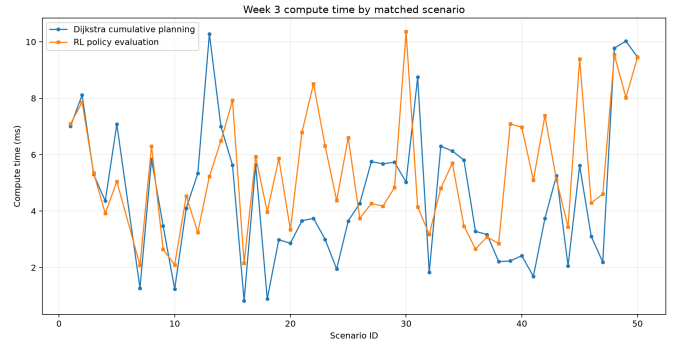


Fig. 4. Compute time by matched scenario. Dijkstra time is cumulative replanning time; RL time is policy evaluation time.

The study therefore supports a trade-off framing. Dijkstra provides optimality and reliability when the current graph is known, but it must replan after changes. RL provides policy-based adaptation, but its learned decisions can be suboptimal and can fail. The current results favor Dijkstra at this scale while leaving open the possibility that RL becomes more attractive as environment size and complexity increase.

A. Limitations

This study has several limitations. First, the grid is small. A 15×15 environment is useful for controlled development, but it may not expose the full computational burden of repeated graph search. Second, static obstacle density is set to zero in the paired dynamic evaluation. This ensures fairness between the two environments but makes the benchmark simpler than a cluttered UAV routing domain. Third, the dynamic setup uses only three fixed toggle cells. Fourth, the RL policy has one failed scenario, which should be treated as a real limitation rather than ignored.

Fifth, compute-time measurements at millisecond scale can be sensitive to implementation details, hardware, and measurement method; the compute-time result here should be read as a small-scale empirical measurement rather than a universal statement about Dijkstra and RL inference speed.

Future work should evaluate larger grids, reintroduce matched static obstacles, vary dynamic obstacle frequency,

compare against additional classical methods such as A*, D* Lite, or Lifelong Planning A*, and test additional RL variants or larger-scale models. A richer benchmark incorporating energy budgets, no-fly zones, and partial observability would also better reflect real UAV routing constraints.

VII. CONCLUSION

This paper compared naive Dijkstra replanning and a dynamically trained RL policy for UAV routing in a shared dynamic grid environment. The evaluation used 50 matched scenarios with identical start-goal pairs and shared dynamic obstacles. Dijkstra succeeded in all scenarios, while RL succeeded in 49. On paired successful scenarios, Dijkstra produced significantly shorter or equal paths, with a Wilcoxon signed-rank result of $W = 0$ and $p = 5.64 \times 10^{-7}$. Compute-time differences were not statistically significant at $p < 0.05$.

On the static 40-scenario evaluation, Dijkstra succeeded on every scenario while DQN succeeded on 97.5%, with a mean path-cost penalty of 0.92 units above optimal. Static compute time also revealed a clear asymmetry: Dijkstra's single graph search per episode was roughly four times faster than DQN's per-step forward-pass cost accumulated over a route. This asymmetry narrows in the dynamic setting, where Dijkstra must itself replan repeatedly, which is why the dynamic compute-time gap is not statistically significant even though the static gap is substantial.

The results show that, for the current 15×15 setup, Dijkstra remains superior in reliability and path optimality. The RL policy demonstrates high adaptability but does not outperform Dijkstra. Future work should test whether RL's deployment-time advantages become more visible in larger and more complex dynamic environments.

REFERENCES

- [1] S. Ghambari, M. Golabi, L. Jourdan, J. Lepagnot, and L. Idoumghar, "Uav path planning techniques: a survey," *RAIRO - Operations Research*, vol. 58, no. 4, pp. 2951–2989, 2024.
- [2] N. Johnson, S. Shafaei, A. Kareem, and S. Sarkar, "A survey of risk-calibrated certifiably safe and resource-aware (rcsr) path planning for unmanned aerial vehicles," *Drones*, vol. 10, no. 5, p. 351, 2026.
- [3] A. Mannan, M. S. Obaidat, K. Mahmood, A. Ahmad, and R. Ahmad, "Classical versus reinforcement learning algorithms for unmanned aerial vehicle network communication and coverage path planning: A systematic literature review," *International Journal of Communication Systems*, vol. 36, no. 5, p. e5423, 2023.
- [4] G.-T. Tu and J.-G. Juang, "Uav path planning and obstacle avoidance based on reinforcement learning in 3d environments," *Actuators*, vol. 12, no. 2, p. 57, 2023.
- [5] Y. Guo and Z. Liu, "Uav path planning based on deep reinforcement learning," *International Journal of Advanced Network, Monitoring and Controls*, vol. 8, no. 3, pp. 81–88, 2023.
- [6] S. Venu and M. Gurusamy, "A comprehensive review of path planning algorithms for autonomous navigation," *Results in Engineering*, vol. 28, p. 107750, 2025.
- [7] J. Zhao, Z. Gan, J. Liang, C. Wang, K. Yue, W. Li, Y. Li, and R. Li, "Path planning research of a uav base station searching for disaster victims' location information based on deep reinforcement learning," *Entropy*, vol. 24, no. 12, p. 1767, 2022.
- [8] L. G. S. d. Rocha, K. A. Q. Caldas, M. H. Terra, F. Ramos, and K. C. T. Vivaldini, "Dynamic q-planning for online uav path planning in unknown and complex environments," 2024.